

I N D E X



Name Annapurna Prasad

Branch

Class

CE8701

Sem

4th

Section

Red Sem

Roll No.

P24013062008

Year 2025-26

Subject Database Management System Lab Record

Sl. No.	Experiment Description	Experiment Date	Submission Date	Remarks Signature
1.	To install MySQL DBMS and understand its basic components.	30/12/25	01/01/26	M. Subudhi 2/1/2026
2.	Create a basic database and perform simple operations.	01/01/2026	06/01/2026	M. Subudhi 6/1/2026
3.	To study and implement various datatype in SQL.	06/01/2026	08/01/2026	M. Subudhi 8/1/2026
4.	To study and implement various data constraints in SQL.	08/01/2026	13/01/26	M. Subudhi 13/1/2026
5.	To write SQL queries for table creation & modification.	13/01/26	16/01/26	M. Subudhi 16/1/2026
6.	To demonstrate the use of various DML operations.	16/01/2026	27/01/26	
7.	To demonstrate the use of various DML operations.	27/01/26	03/02/26	

[illegible]

Experiment No-01

Aim of the experiment:- To install MySQL Database Management system and understand its basic components.

Requirements:-

Hardware:- Personal Computer / Laptop with at least 4GB RAM.

Software:- Windows 10/11, Linux or mac OS.

Network:- Internet connections for downloading MySQL installer.

Disk Space:- Minimum 2GB free space for installation.

Procedure:-

Step-1: Download MySQL

Visit the official MySQL website.

Download MySQL Community Server

Choose the appropriate installer for your operating system.

Step-2: Run the Installer

- Execute the downloaded MySQL installer.
- Choose installation type:-
 - Developer Default: includes MySQL Server, Workbench, and development tools
 - Server Only: Only MySQL ~~Server~~ Server
 - Custom: Select specific components

Step-3: Configuration

- Server Configuration Type:- Choose "Development Computer" for learning purposes.
- Port Number:- Default port 3306 (change if needed)
- Authentication:- Use strong Password Encryption
- Root Password:- Set a strong password for the root user.

Step-4: Complete Installation

- Apply configuration settings

Start MySQL Server as a Windows Service

- Complete the Installation process

Step-5: Verify Installation

- Open MySQL Command Line Client
 - Login with root credentials
 - Execute basic commands to verify functionality.
- * Post-Installation Commands

```
SELECT VERSION();
```

```
SELECT NOW();
```

```
SHOW DATABASES;
```

```
CREATE DATABASE test_db;
```

```
USE test_db;
```

```
CREATE TABLE test_table (  
    id INT PRIMARY KEY,  
    name VARCHAR(50).
```

```
);
```

```
INSERT INTO test_table VALUES (1, 'Test Record');
```

```
SELECT * FROM test_table;
```

```
DROP TABLE test_table;
```

```
DROP DATABASE test_db;
```

* Common Installation Issues and Solutions

- Port 3306 already in use: Change port number or stop conflicting services.
- Service won't start: Check Windows Services and restart MySQL service.
- Login denied: Verify username/password and authentication method.
- Connection refused: Ensure MySQL service is running.

* Conclusion:-

Successfully installed MySQL Database Management System and verified its basic components and functionality.

Anubhuti
2/1/2026

Submitted by:-

Name:- Annapurna Prasad

Branch:- Computer & IOT

Regd No:- F2401306 2008

Semester:- ^{4th} ~~3rd~~ Sem

Date of Submission:- 01/01/26

Teacher's Signature.....

Experiment No-02

* Aim of the experiment:- Create a ^{basic} database and perform simple operations.

* Basic Syntan and description:-

• Create database :- This statement is used to create a new SQL database.

CREATE DATABASE database_name;

• Use the database :- This statement is used to use the available database.

USE database_name;

• Delete the database :- This statement is used to drop an existing SQL database.

DROP DATABASE database_name;

* Example Script :-

CREATE DATABASE CollegeDB;

USE CollegeDB;

DROP DATABASE CollegeDB;

* Conclusion:-

Successfully created a basic database and performed simple operations.

Submitted by:-

Name:- Annapurna Prasad Branch:- C.E.
IT

Regd No:- F24013062008 D.O.S = 06/01/26

Teacher's Signature.....

Mubudhi
6/1/2026.

Experiment No-03

* Aim of the experiment :- To study and implement various datatype in SQL.

* Theory :-

Different Data Types :-

• Numeric Data Types :-

• INT : Integer

• DECIMAL(p,s) :- Fixed-point numbers with precision(p) and scale (s).

• FLOAT :- Floating-point numbers

• String Data Types :-

• CHAR(n) :- Fixed-length string of length (n).

• VARCHAR(n) :- Variable-length string with a maximum length of n.

• TEXT :- Large text data

• BLOB :- Binary Large Object for storing binary data.

• LONG BLOB :- Large Binary Large Object for storing very large binary data.

• ENUM :- Enumeration, a string object with a predefined set of values.

Table :-

Field	Type	NULL	Key	Default	Extra
Product ID	int	YES		NULL	
Price	decimal(10,2)	YES		NULL	
Weight	float	YES		NULL	
Product Code	char(5)	YES		NULL	
Product Name	varchar(100)	YES		NULL	
Description	text	YES		NULL	
Product Image	blob	YES		NULL	
Large Image	longblob	YES		NULL	
Category	enum('Electronics', 'Clothing', 'Furniture')	YES		NULL	
Mfg Date	date	YES		NULL	
Mfg Time	time	YES		NULL	
Mfg Year	year	YES		NULL	

- Date and Time Datatypes:-

- DATE : Date Values (YYYY-MM-DD)
- TIME : Time Values (HH:MM:SS)
- YEAR : Year Values (YYYY)

- * Commands:-

```
CREATE TABLE ProductInventory (  
    ProductID INT,  
    Price DECIMAL(10,2),  
    Weight FLOAT,  
    ProductCode CHAR(5),  
    ProductName VARCHAR(100),  
    Description TEXT,  
    ProductImage BLOB,  
    LargeImage LONGLOB,  
    Category ENUM('Electronics', 'Clothing', 'Furniture'),  
    MfgDate DATE,  
    MfgTime TIME,  
    MfgYear YEAR,  
);  
DESCRIBE ProductInventory;
```


* Conclusion :-

Successfully studied and implemented various SQL data types.

✓
Submitted
8/1/2026

Submitted By :-

Name :- Annapurna Prasad

Branch :- Computer Engg & IT

Regd No :- F24013062008

Semester :- ^{4th} ~~2nd~~ Semester

Date of Submission :- 08/01/26

Experiment No:- 04

- * Aim of the experiment:- To study and implement various data constraints in SQL.
- * Theory:-
 - NOT NULL:- This constraint enforces a column to NOT accept NULL values.
column name datatype NOT NULL
 - UNIQUE:- This constraint ensures that all values in a column are different.
column name datatype ~~PRIMARY KEY~~ UNIQUE
 - PRIMARY KEY:- This constraint uniquely identifies each record in a table.
column name datatype PRIMARY KEY
 - FOREIGN KEY:- This constraint is used to link two tables together.
FOREIGN KEY (column name) REFERENCES parent table (parent column)
 - CHECK:- This constraint ensures that all values in a column satisfy a specific condition.
column name datatype CHECK (condition)

Table :-

Field	Type	Null	Key	Default	Extra
EmployeeID	int	NO	PRI	NULL	
FirstName	varchar(50)	NO		NULL	
LastName	varchar(50)	NO		NULL	
Email	varchar(100)	YES	UNI	NULL	
Age	int	YES		NULL	
DepartmentID	int	YES	MUL	NULL	
Salary	decimal(10,2)	YES		30000.00	

- DEFAULT:- Default constraint is used to set a default values for a column.

Syntax:-

(i) Not NULL:-

at create :- CREATE TABLE table_name (
column_name datatype NOT NULL
);

at alter :- ALTER TABLE table_name
MODIFY column_name datatype NOT NULL;

(ii) UNIQUE:-

at create :- CREATE TABLE table_name (
column_name datatype UNIQUE
);

at alter :- ALTER TABLE table_name
ADD UNIQUE (column_name);

drop :- ALTER TABLE table_name
DROP INDEX constraint_name;

(iii) PRIMARY KEY:-

at create :- CREATE TABLE table_name (
column_name datatype PRIMARY KEY
);

Teacher's Signature.....

at alter:- ALTER TABLE table_name
ADD PRIMARY KEY (column_name);

drop:- ALTER TABLE table_name
DROP PRIMARY KEY;

(iv) FOREIGN KEY:-

at create:- CREATE TABLE table_name (
column_name data_type,
FOREIGN KEY (column_name) REFERENCES
parent_table (parent_column));

at alter:- ALTER TABLE table_name
ADD CONSTRAINT FK_name FOREIGN KEY (column_name)
REFERENCES parent_table (parent_column);

drop:- ALTER TABLE table_name
DROP FOREIGN KEY FK_name;

(v) CHECK:-

at create:- CREATE TABLE table_name (
column_name datatype CHECK (condition));

at alter:- ALTER TABLE table_name
ADD CHECK (condition);

drop:- ALTER TABLE table_name
DROP CONSTRAINT constraint_name;

Teacher's Signature.....

DEFAULT :-

at create:- CREATE TABLE table_name (
column_name datatype DEFAULT default_value);

at alter:- ALTER TABLE table_name

ALTER column_name SET DEFAULT default_value;

drop:- ALTER TABLE table_name

ALTER column_name DROP DEFAULT;

Commands :-

```
CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50) NOT NULL
);
```

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    FirstName VARCHAR(50) NOT NULL,
    LastName VARCHAR(50) NOT NULL,
    Email VARCHAR(100) UNIQUE,
    Age INT CHECK (Age > 18),
    DepartmentID INT,
    Salary DECIMAL(10,2) DEFAULT 30000.00,
    FOREIGN KEY (DepartmentID) REFERENCE
        Departments (DepartmentID)
);
```

DESCRIBE Employees;

Teacher's Signature.....

Conclusion:-

Successfully studied and implemented SQL data constraints.

Submitted By

Name:- Annapurna Prasad

Branch:- Computer Engg & IOT

Regd No:- F2401306 2008

Semester:- ^{4th} ~~3rd~~ Sem

Date of Submission:- 13/01/26

Experiment No-05

Aim of the experiments:- To write SQL queries for table creation and modification.

Theory:-

CREATE TABLE:

```
CREATE TABLE table_name (
    column1 datatype,
    column2 datatype,
    ....
);
```

ALTER TABLE:

ADD COLUMN:

```
ALTER TABLE table_name
ADD column_name datatype;
```

MODIFY COLUMN:

```
ALTER TABLE table_name
MODIFY column_name new_datatype;
```

DROP COLUMN:

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

RENAME TABLE:

RENAME TABLE ~~old table name~~ To new-table-name;

TRUNCATE TABLE:

TRUNCATE TABLE table-name;

DROP TABLE:

DROP TABLE table-name;

Commands :-

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    FirstName VARCHAR (50),
    LastName VARCHAR (50),
    Age INT,
    Department VARCHAR (50)
);
```

```
ALTER TABLE Employees
ADD Email VARCHAR (100);
```

```
ALTER TABLE Employees
MODIFY Age SMALLINT;
```

```
ALTER TABLE Employees
DROP COLUMN Department;
```

Teacher's Signature.....

RENAME TABLE Employees To Staff;

TRUNCATE TABLE Staff;

DROP TABLE Staff;

* Conclusion:-

Successfully implemented Data Definition Language (DDL) commands.

✓ Asubndhi
16/11/2026.

Submitted By:-

Name:- Annapurna Prasad

Branch:- Computer Engg & IT

Regd No:- F24013062008

Semester:- ^{4th} ~~3rd~~ Semester

Date of Submission:- 16/01/26

Experiment No-06

* Aim of the experiment:-

To demonstrate the use of various DML operations i.e., insertion, deletion and updating.

* Theory:-

• INSERT INTO :- The INSERT INTO statement is used to insert new records in a table.

Syntax:- INSERT INTO table_name (column1, column2, column3,)
VALUES (value1, value2, value3,);

• DELETE :- The DELETE statement is used to delete existing records in a table.

Syntax:- DELETE FROM table_name WHERE condition;

• UPDATE :- The UPDATE Statement is used to modify the existing records in a table.

Syntax:- UPDATE table_name
SET column1 = value1, column2 = value2,
WHERE condition;

After Insertion

EmployeeID	FirstName	LastName	Age	Department
1	John	Doc	30	HR
2	Jane	Smith	25	IT

After Deletion

EmployeeID	FirstName	LastName	Age	Department
2	Jane	Smith	25	IT

After Updating

EmployeeID	FirstName	LastName	Age	Department
2	Jane	Smith	31	IT

* Commands :-

```
INSERT INTO Employees (EmployeeID, FirstName,  
VALUES (                LastName, Age, Department)  
VALUES(1, 'John', 'Doe', 30, 'HR'),  
VALUES(2, 'Jane', 'Smith', 25, 'IT');
```

```
SELECT * FROM Employees;
```

```
DELETE FROM Employees  
WHERE EmployeeID = 1;
```

```
SELECT * FROM Employees;
```

```
UPDATE Employees  
SET Age = 31  
WHERE EmployeeID = 2;  
SELECT * FROM Employees;
```

* Conclusion:-

Successfully implemented insertion, deletion and updating (DML) commands.

Submitted By:-

Name:- Annapurna Prasanna

Branch:- Computer Engg. & IT

Regd No:- F24013062008

Semester:- 4th Semester.

Date of Submission:-

Experiment No-07

- * Aim of the experiment :- To demonstrate the use of various DML operations i.e., SELECT statement, WHERE clause, AND, OR, NOT operators, ORDER BY, and INSERT SELECT statement.

* Theory :-

- SELECT statement :- The SELECT statement is used to retrieve data from a database. It allows the user to specify which columns to retrieve and from which table.

Syntax

```
SELECT column1, column2, .....  
FROM table_name;
```

To select all columns from a table :-

```
SELECT * FROM table_name;
```

- WHERE Clause :- The WHERE clause is used to filter records that meet a certain condition. It is used in conjunction with the SELECT, UPDATE, DELETE etc. statements.

Syntax

```
SELECT * FROM table_name  
WHERE condition;
```

© PLUX

- **AND, OR, NOT Operators:-** The AND, OR and NOT operators are used to combine multiple conditions in a WHERE clause.

Syntax:-

```
SELECT * FROM table_name  
WHERE condition1 AND condition2;
```

```
SELECT * FROM table_name  
WHERE condition1 OR condition2;
```

```
SELECT * FROM table_name  
WHERE NOT condition;
```

- **ORDER BY clause:-** The ORDER BY clause is used to sort the result set in either ascending (ASC) to descending (DESC) order.

Syntax:-

```
SELECT * FROM table_name  
ORDER BY column_name ASC|DESC;
```

- **INSERT SELECT statement:-** The INSERT SELECT statement is used to insert new records into a table by selecting data from another table.

Syntax:-

```
INSERT INTO table_name (column1, column2, .....  
SELECT column1, column2, .....  
FROM another_table  
WHERE condition;
```

Before Insertion

EmployeeID	FirstName	LastName	Age	Department
1	John	Doe	28	IT
2	Jane	Smith	35	HR
3	Bob	Johnson	42	Finance
4	Alice	Brown	29	IT

After Insertion

EmployeeID	FirstName	LastName	Age	Department
1	John	Doe	28	IT
2	Jane	Smith	35	HR
3	Bob	Johnson	42	Finance
4	Alice	Brown	29	IT
5	Charlie	Wilson	27	HR
6	Diana	Davis	32	Sales

* Commands :-

```
SELECT * FROM Employees;  
WHERE Age > 30 AND Department = 'HR'
```

```
ORDER BY Lastname ASC;
```

```
INSERT INTO Employees (EmployeeID, FirstName,  
                        LastName, Age, Department)
```

```
SELECT EmployeeID, FirstName, LastName, Age, Department  
FROM AnotherEmployees  
WHERE Age > 25;
```

```
SELECT * FROM Employees;
```

* Conclusion :-

Successfully implemented SELECT statement, WHERE clause, AND, OR, NOT operators, ORDER BY clause, and INSERT SELECT statement (DML) commands.

Submitted By :-

Name :- Annapurna Prasad

Branch :- Computer Engg & IOT

Regd No :- F24013062008

Semester :- 4th Semester.

Date of Submission :-

Experiment No-08

* Aim of the experiments :- To demonstrate the use of aggregate functions i.e., min, max, count, sum and average.

* Theory :-

Aggregate functions perform a calculation on a set of values and return a single value. They are often used with the GROUP BY clause in SQL to group the result set by one or more columns.

(i) Min and Max :- The MIN() function returns the smallest value in a set, while the MAX() function returns the largest value.

SELECT MIN(column_name) FROM table_name;

SELECT MAX(column_name) FROM table_name;

(ii) Count, Sum, and Average :- The COUNT() function returns the number of rows that match a specified condition, the SUM() function returns the total sum of a numeric column, and the AVG() function returns the average value of a numeric column.

SELECT COUNT(column_name) FROM table_name;

SELECT SUM(column_name) FROM table_name;

SELECT AVG(column_name) FROM table_name;

Commands:-

```
SELECT MIN(salary) AS MinimumSalary FROM Employees;  
SELECT MAX(salary) AS MaximumSalary FROM Employees;  
SELECT COUNT(*) AS TotalEmployees FROM Employees;  
SELECT SUM(salary) AS TotalSalary FROM Employees;  
SELECT AVG(salary) AS AverageSalary FROM Employees;
```

Conclusion:-

Successfully demonstrated the use of various aggregate functions in SQL, including MIN(), MAX(), COUNT(), SUM(), and AVG().

Submitted By:-

Name:- Annapurna Prasad

Branch:- Computer Engg. & IT

Regd No:- F24013062008

Semester:- 4th Semester

Date of Submission:-

Experiment No-09

* Aim of the experiment :- To demonstrate the use of aggregate functions i.e., like, wildcards, in, and between.

* Theory :-

Aggregate functions performs a calculation on a set of values and return a single value. They are often used with the GROUP BY clause in SQL to group the result set by one or more columns.

(i) Like :- The LIKE operator is used in a WHERE clause to search for a specified pattern in a column.

TAX `SELECT column1, column2, .....`

`FROM table_name`

`WHERE column LIKE pattern;`

(ii) Wildcards :- The % wildcard represents zero or more characters, while the _ wildcard represents a single character.

X `SELECT column_name(s) FROM table_name WHERE`
`column_name LIKE 'a%';`

`SELECT column_name(s) FROM table_name WHERE`
`column_name LIKE '_a%';`

(iii) In:- In IN operator allows you to specify multiple values in a WHERE clause.

Syntax SELECT column_name(s) FROM table_name
WHERE column_name IN (value 1, value 2,);

(iv) Between:- The BETWEEN operator selects values within a given range. The values can be numbers, text or dates.

SELECT column_name(s) FROM table_name
WHERE column_name BETWEEN value 1 ~~and~~ AND value 2;

* Commands:-

SELECT * FROM Employees WHERE name LIKE 'J%';
SELECT * FROM Employees WHERE name LIKE '_a%';
SELECT * FROM Employees WHERE department IN
('HR', 'Finance');

SELECT * FROM Employees WHERE salary BETWEEN
50000 AND 100000;

* Conclusion:-

Successfully demonstrated the use of various aggregate functions in SQL, including LIKE, wildcards, IN and BETWEEN.

Submitted By:-

Name:- Annapurna Prasad

Branch:- Computer Engg & IT

Regd No:- F24013062008

Semester:- 4th Sem

Date of Submission:-